



Unidade 4 — Por que desenvolver seu próprio Framework?



Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Implementar** um Router do zero;
- **Extrair** parâmetros de URLs dinâmicas;
- **Analisar** como o Express.js implementa roteamento;
- **Comparar** nossa implementação com a do Express.

Taxonomia de Bloom: Implementar, Extrair, Analisar, Comparar.



Estrutura de Dados do Router

```
// Cada rota é um objeto:  
{  
  method: 'GET',  
  path: '/api/usuarios/:id',  
  handler: function(req, res) { ... },  
  paramNames: ['id'],  
  pattern: /^\/api\/usuarios\/([\/]+)$/`  
}
```

- **method:** GET, POST, PUT, DELETE;
- **path:** o padrão da URL (com parâmetros);
- **handler:** a função que processa a requisição;
- **paramNames:** nomes dos parâmetros (:id);
- **pattern:** regex para matching da URL.



Implementação do Router

```
class Router {
  constructor() {
    this.routes = [];
  }

  add(method, path, handler) {
    const paramNames = [];
    // Converter :param para grupos de regex
    const pattern = path.replace(/:([w+])/g, (_, name) => {
      paramNames.push(name);
      return '([^/]+)';
    });
    this.routes.push({
      method: method.toUpperCase(),
      path,
      handler,
      paramNames,
      pattern: new RegExp(`^${pattern}$`)
    });
  }

  match(method, url) {
    for (const route of this.routes) {
      if (route.method !== method.toUpperCase()) continue;
      const match = url.match(route.pattern);
      if (match) {
        // Extrair valores dos parâmetros
        const params = {};
        route.paramNames.forEach((name, i) => {
          params[name] = match[i + 1];
        });
        return { handler: route.handler, params };
      }
    }
    return null;
  }
}
```





Exemplo de Uso

```
const router = new Router();

// Registrar rotas
router.add('GET', '/api/usuarios', (req, res) => {
  res.json(usuarios);
});

router.add('GET', '/api/usuarios/:id', (req, res) => {
  const usuario = usuarios.find(u => u.id == req.params.id);
  res.json(usuario);
});

router.add('POST', '/api/usuarios', (req, res) => {
  usuarios.push(req.body);
  res.status(201).json(req.body);
});

// Simular uma requisição
const result = router.match('GET', '/api/usuarios/42');
console.log(result.params); // { id: "42" }
result.handler(req, res); // Executa o handler
```



Síntese da Aula

Conceitos principais:

1. O Router mapeia **método + URL** para **handler**;
2. Parâmetros de URL são extraídos com **regex**;
3. O método **match()** encontra a rota correspondente;
4. A implementação é **simples** — menos de 30 linhas;
5. O Express usa o **mesmo princípio** internamente.

Pergunta reflexiva:

Se o Router é tão simples, por que o Express tem milhares de linhas de código?

Próxima aula: Middleware e Pipeline — Vamos implementar o sistema de middlewares.

